PBDB2 API — Response Envelope: Design Comparison

Status: Design rationale (under discussion) **Audience:** PBDB2 contributors and stakeholders **Subject:** Choosing the JSON response envelope for the new PBDB2 REST API

---

1. Context

PBDB2 is a ground-up redevelopment of the Paleobiology Database. The new backend is PostgreSQL, with each resource row storing its domain content as a **JSONB payload** alongside relational metadata. Two properties of the new schema are central to everything below:

- **Stable identity.** Every record carries a `permid` — a permanent identifier that stays constant across edits. Internal serial primary keys are never exposed; `permid` is the public handle for a record.
- **Immutable versioned lineage.** Records are never updated in place. An edit inserts a new row that *succeeds* the prior one, forming a version chain. The "current" record is the head of that chain. Deletion is a soft flag, not a removal.

The new API is being built on Fastify (JavaScript, ES modules). A key downstream goal is to **generate OpenAPI / Swagger documentation directly from the project's JSON Schema definitions**, which Fastify validates against natively.

This document compares two candidate response formats:

1. **The proposed PBDB2 envelope** — a pragmatic `{ data, meta, links }` structure (referred to as "the proposed envelope" below).
2. **The legacy PBDB `data1.2` envelope** — the format served by the current, long-standing Paleobiology Database data service.

The recommendation is to adopt the proposed envelope. Sections 2–5 lay out the reasoning so it stands on its own.

---

2. The proposed envelope

A single, consistent structure for every endpoint:

- **`data`** — the resource itself (an object for a single record; an array for a collection). This is the typed domain payload and nothing else.
- **`meta`** — everything *about* the record that isn't the record: its type, soft-delete status, version lineage, provenance, and (for lists) result counts and timing.
- **`links`** — URLs: the record's own address (`self`), related resources, the version history, and pagination (`next` / `prev`) for lists.

Single record

`GET /api/v1/collections/5f3c8e9a-…`

```
{
  "data": {
    "permid": "5f3c8e9a-…",
    "name": "Anza-Borrego",
    "country": "US",
    "state": "California"
  },
  "meta": {
    "type": "collection",
    "removed": false,
    "version": { "supersedes": "1b2d-…", "supersededBy": null },
    "enterer": "42",
    "authorizer": "7"
  },
  "links": {
    "self": "/api/v1/collections/5f3c8e9a-…",
    "primaryReference": "/api/v1/references/9a1f-…",
    "earlyInterval": "/api/v1/intervals/…",
    "history": "/api/v1/collections/5f3c8e9a-…/versions"
  }
}
```

Collection (list)

`GET /api/v1/collections?…`

```
{
  "data": [ { "permid": "5f3c8e9a-…", "name": "Anza-Borrego" }, { "…": "…" } ],
```

```
  "meta": {
    "found": 1240,
    "returned": 100,
    "elapsed_ms": 18
  },
  "links": {
    "self": "/api/v1/collections?…",
    "next": "/api/v1/collections?…",
    "prev": null
  }
}
```

The shape never changes: a client parser written once keeps working across every resource and every query.

---

3. The legacy PBDB `data1.2` envelope

The current Paleobiology Database data service returns a flat envelope in which result metadata sits as sibling keys alongside the records, and domain fields use compact three-letter codes by default.

```
GET /data1.2/colls/single.json?id=1003&show=loc
```

```
{
  "elapsed_time": 0.012,
  "records_found": 1,
  "records_returned": 1,
  "records": [
    {
      "oid": "col:1003",
      "rid": "ref:551",
      "nam": "Anza-Borrego",
      "ein": "txn:43381",
      "cc2": "US",
      "stp": "California"
    }
  ]
}
```

Notable characteristics:

- Domain fields are abbreviated (`oid`, `rid`, `nam`, `ein`) unless the caller requests a full vocabulary via a query parameter.
- Identifiers are type-prefixed strings (`col:1003`, `ref:551`, `txn:43381`).
- Result metadata (`records_found`, `elapsed_time`) is interleaved with the data at the top level.
- Optional blocks (data provenance, license, the echoed parameters, warnings) can be toggled on with query parameters — and doing so can change the envelope, including the name of the records array.
- A single record is still returned inside an array.
- Output format is selected by a URL suffix (`.json`, `.csv`, `.tsv`).

This format has served the database well for many years. The comparison that follows is not a criticism of it on its own terms — it is an assessment of fit for the *new* PBDB2 data model and goals.

---

4. Side-by-side comparison

Dimension	Legacy `data1.2`	Proposed envelope	Why the proposed form fits PBDB2
Field names	Three-letter codes by default; full names only on request	Full descriptive names, always	Self-documenting. The compact codes required a lookup table and existed to save bytes on the wire — a concern largely eliminated by standard HTTP compression.
Identity	Type-prefixed strings (`col:1003`)	Clean `permid` plus `meta.type`	`permid` is the database's native stable identifier. Type is recorded in one predictable place rather than embedded in every identifier string.
Metadata placement	Flat sibling keys mixed with data	Isolated in `meta`	Data and housekeeping never collide; clients read `data` without filtering out bookkeeping fields.

Dimension	Legacy `data1.2`	Proposed envelope	Why the proposed form fits PBDB2
Envelope stability	Shape shifts with query parameters (including the records array name)	One shape, always	A parser written once continues to work. A parameter-dependent shape is a recurring source of client bugs.
Relationships	Implicit — inferred by parsing id prefixes and knowing the routes	Explicit `links`, including version `history`	Discoverable, and the only place that can express the version lineage at all.
Single vs. list	A single record is an array of one	Single is an object; list is an array	Matches HTTP intent; no unwrapping ritual.
Errors	Frequently `200 OK` with a `warnings` array	Proper HTTP status codes with a structured error body	Standard client tooling, retries, and monitoring all key off status codes.
Format negotiation	URL suffix (`.json` / `.csv`)	HTTP `Accept` header	Content negotiation is the standards-based mechanism; suffixes are a workaround from a server-templating era.
Versioning & soft delete	No representation	First-class in `meta` and `links`	The PBDB2 schema is built on version lineage and soft deletion; the legacy envelope has nowhere to put either.
Generated API docs	Hand-maintained	Generated; `data` references the project's JSON Schema directly	A stated PBDB2 goal. The compact-vocabulary, parameter-dependent legacy shape is difficult to express cleanly in OpenAPI.

---

5. Recommendation and rationale

Adopt the proposed `{ data, meta, links }` envelope.

The legacy envelope was engineered for constraints that no longer apply: terse field codes to reduce payload size before compression was routine, URL-suffix format selection from a server-templating world, flat metadata because there was no separation discipline, and an envelope whose shape varies with the request. None of it accommodates the two properties the PBDB2 database is actually built around — **stable `permid` identity** and **immutable versioned lineage**. The legacy format has no slot for either.

The proposed envelope is therefore not a cosmetic reskin. It is the representation shaped by the new data model:

- `data` carries the JSONB payload as-is, so it can be validated against — and documented from — the project's JSON Schema definitions with no translation layer.
- `meta` gives version lineage, soft-delete status, and provenance a permanent, predictable home.
- `links` makes relationships and history explicit and navigable.

Carrying the legacy envelope forward would mean bolting the new database's core concepts onto the side of a format designed never to need them.

What is *not* being abandoned

The case for moving forward is strengthened, not weakened, by naming the legacy service's genuine obligations. These are **migration concerns**, separable from the envelope decision:

1. **A compatibility layer.** If existing consumers depend on the `data1.2` format, that can be provided later as a thin translation layer in front of the new API — it need not constrain the new design.
2. **CSV / TSV export.** Researchers rely on tabular export. The proposed envelope supports this through `Accept`-header content negotiation and a serializer; it remains on the roadmap.
3. **Citable identifiers.** Identifiers cited in publications under the legacy scheme will warrant a mapping to `permid` over time.

In short: the new API keeps the legacy service's real responsibilities as migration line items, while the core representation moves to a format built for this database rather than the previous one.

Appendix: open sub-decisions

These do not affect the envelope's shape and can be settled independently:

- **Pagination mechanism (cursor vs. offset).** Deferrable. The envelope already reserves `links.next` / `links.prev` and the `meta` counts, so the mechanism can be chosen later by changing only query parameters and the underlying query — with no change to the response shape. For large resources, cursor-based pagination is the expected choice because deep offset pagination degrades on large tables.